2024/2025	AP – PROJET 1
BTS SIO 1SIOB	Auteur : Groupe 5
	Date de rédaction : 15/09/2025

Compte-rendu de TP

Itération n°1 – Ticket n°2 – étude de l'existant

Table des matières

I - Rappel du contexte 1
II - Étude des fonctionnalités existantes 1
III - Architecture MVC en PHP 3
III.1 Analyse de l'affichage de la liste des restaurants 3
III.2 Analyse d'un script du dossier test 5
III.3 Les couches « vue et « modèle » 7
III.4 La couche contrôleur 8
III.4.1 - Le contrôleur listeRestos.php 8
III.4.2 - Le contrôleur detailResto.php 9
III.4.3 - Synthèse 9
IV - Synthèse globale 10

I - Rappel du contexte

R3st0.fr est un site web de critique de restaurants. À l'image des sites de ce type il a pour vocation le recensement des avis des consommateurs et la diffusion de ces avis aux visiteurs.

Ce site web est développé en PHP en suivant le patron de conception "modèle vue contrôleur" (pattern MVC). L'architecture retenue pour ce projet permet d'appréhender la programmation web d'une manière structurée.

L'objectif de ce document est d'analyser de manière globale l'organisation du site.

Afin de mieux voir l'objectif du site, et les différentes fonctionnalités, le site final est consultable en interne à l'adresse suivante : https://www3.jolsio.net/test2slam/public/siteresto (adresse interne uniquement)

II - Étude des fonctionnalités existantes

Ressources à utiliser

- le code source du site, sous la forme d'un projet NetBeans, est à télécharger depuis Moodle.

Travail à faire

1. Paramétrer le projet pour que celui-ci soit exécutable sur le serveur web local :

On crée le dossier du projet à partir de celui fourni et on fait en sorte que celui-ci soit copié automatiquement vers le dossier « www » de Wamp pour que le projet puisse s'exécuter sur le serveur web local.

Puis on fait en sorte de pouvoir le lancer avec l'url du projet.

2. Créer la base de données « resto2 » en exécutant les scripts du dossier « sql » du projet :

Avec les codes fournis dans le projet, nous avons créé la Base de Données « resto2 ».

3. Paramétrer l'accès à la base de données par l'application :

Dans le script modele/dao/Bdd.class.php, compléter les lignes suivantes afin d'indiquer les bonnes informations :

private static $login = ""; // login utilisateur de la BDD

private static $mdp = ""; // mdp utilisateur de la BDD

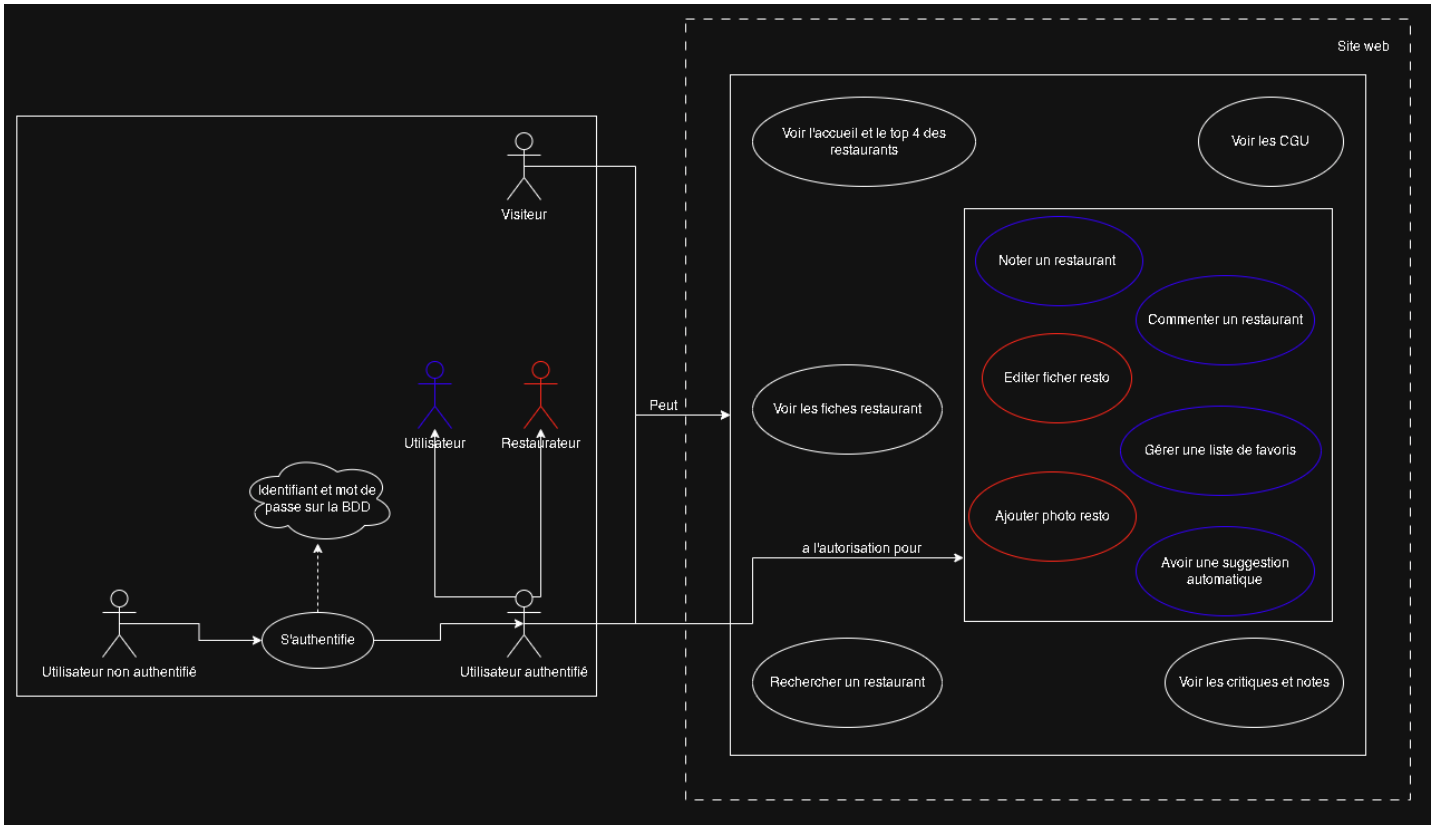
private static $bd = ""; // nom de la BDD

private static $serveur = ""; // nom de domaine du serveur de BDD

Nous avons donc mis les bonnes valeurs dans cette partie du code de connexion du modèle DAO afin d'établir la connexion avec la Base de Données.

```
private static ?PDO $pdo=null;
private static $login = "resto_util";    // login utilisateur de la BDD
private static $mdp = "secret";          // mdp  utilisateur de la BDD
private static $bd = "resto2";           // nom de la BDD
private static $serveur = "localhost";   // nom de domaine du serveur de BDD
private static $pdoOptions = array  (
```

4. Réaliser un diagramme de cas d'utilisation des fonctionnalités existantes



III - Architecture MVC en PHP

Découverte générale du patron de conception MVC.

III.1 Analyse de l'affichage de la liste des restaurants

Travail à faire :

1. A l'aide de la documentation officielle PHP, rappeler le rôle des mots clés suivants : include_once, include, require_once, require, use

Chaque mot clé possède son rôle propre :

- include_once : Inclut et évalue une seule fois le fichier spécifié durant l'exécution du script. Il est utilisé lorsqu'un fichier sera évalué ou inclus plusieurs fois ou pour être sûr qu'il ne sera inclus qu'une seule fois.

- include : Include inclut et exécute le fichier spécifié en argument. Lorsqu'un fichier est inclus, le code le composant hérite de la portée des variables de la ligne où l'inclusion apparaît.

- require_once : PHP vérifie si le fichier a déjà été inclus, et si c'est le cas, ne l'inclut pas une deuxième fois.

- require : Réalise la même chose que include mais en produit une erreur en cas d'erreur alors qu'include produit un avertissement.

- use : use est une fonction anonyme, elle indique à une classe d'hériter d'un trait.

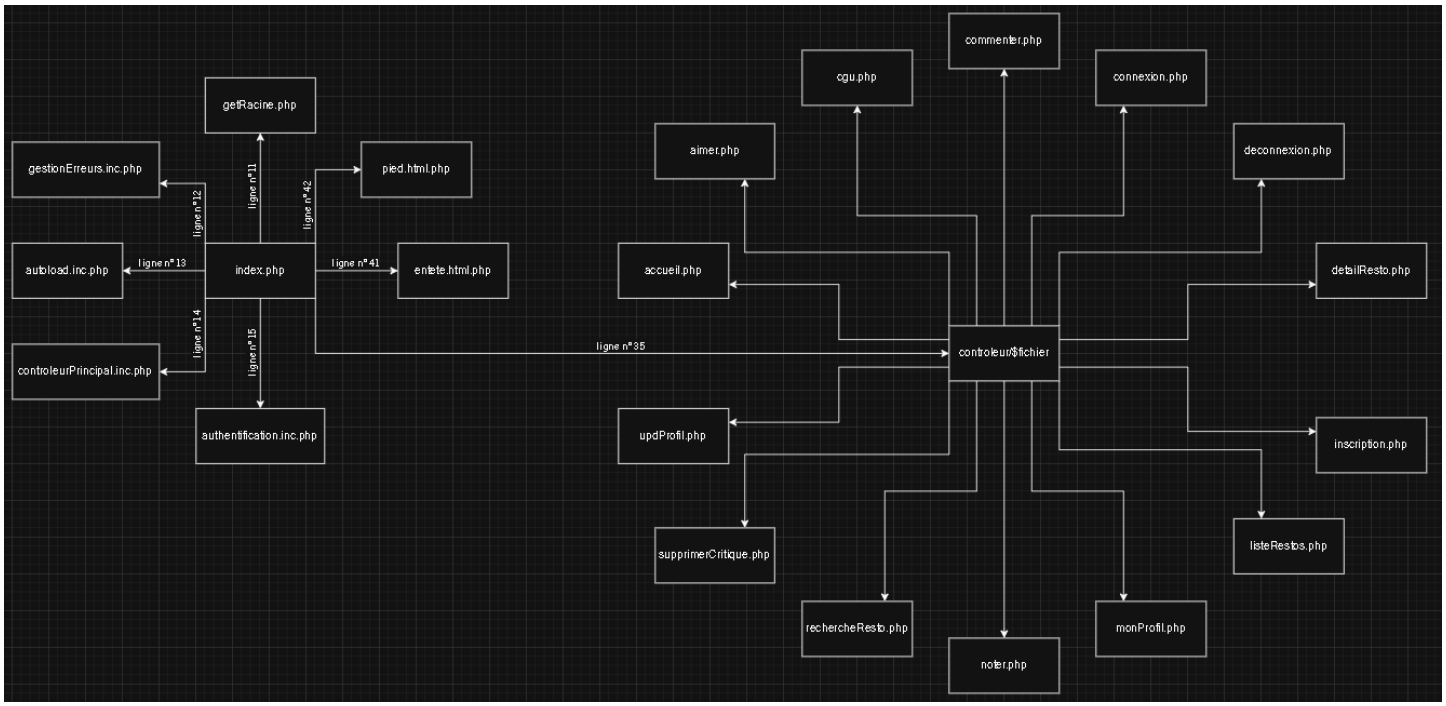
2. Déterminer la valeur de la variable $fichier à la ligne précédant la balise de fin de PHP dans index.php ; moyen : affichage intermédiaire ou bien mode débogage + point d'arrêt

La variable $fichier a pour valeur « accueil.php » à ce moment là du code. Cela est dû au fait que l'index est la page d'accueil et prend donc pour « valeur » accueil.php

3. En partant du fichier index.php, schématiser l'ensemble des fichiers utilisés (require ou require_once) pour afficher la liste des restaurants (URL : index.php?action=liste) ;
Le schéma comportera :

- des rectangles portant le nom de chaque fichier,
- des flèches pointant vers le fichier inclus. Sur la flèche, indiquer le n°de ligne de code de l'inclusion

On part donc de « index.php » et on fait les liaisons vers les différents fichiers utilisés dans les « include_once » du code aux lignes 11, 12, 13, 14, 15, 41 et 42. Pour la ligne 35 en revanche, on inclut l'entièreté du dossier « controleur ». Il faut donc penser à tout les fichiers qui font parti de ce dossier et qui sont eux aussi inclut par cette ligne 35.



4. Après avoir observé le code de la classe RestoDAO.class.php, relever l'ensemble des requêtes SQL contenues dans ses méthodes.

Nom de la méthode	Requête SQL
getOneById	SELECT * FROM resto WHERE idR = :idR
getAll	SELECT * FROM resto
getTop4	SELECT SUM(note) AS NotesCumulees, r.idR, nomR, numAdrR, voieAdrR, cpR, villeR, latitudeDegR, longitudeDegR, descR, horairesR FROM resto r INNER JOIN critiquer c ON r.idR = c.idR GROUP BY r.idR, nomR, numAdrR, voieAdrR, cpR, villeR, latitudeDegR, longitudeDegR, descR, horairesR ORDER BY NotesCumulees DESC LIMIT 4;
getAllByNomR	SELECT * FROM resto WHERE nomR LIKE :nomR

5. Quels sont les points communs de ces requêtes SQL ?

Toutes ces requêtes SQL interrogent la table **resto** pour extraire des informations sur les restaurants. Elles diffèrent seulement par le type de filtrage ou d'agrégation appliqué (sélection brute, par identifiant, par nom, ou avec calculs statistiques).

III.2 Analyse d'un script du dossier test

Cette section n'est exécutée que lorsque l'on veut effectuer un test unitaire, indépendamment de l'affichage du site web.
Exécutez le script suivant : testRestoDAO.php et observez le code correspondant.

Travail à faire :

1. Pour chaque méthode DAO testée, observer le résultat affiché, la requête SQL exécutée, puis expliquer son rôle d'une manière très générale.

```
public static function getOneById(int $id): ?Resto {
    $leResto = null;
    try {
        $requete = "SELECT * FROM resto WHERE idR = :idR";
```

Exécute une requête SQL pour récupérer un enregistrement de la table resto correspondant à l'identifiant fourni. Si trouvé, il est transformé en objet Resto.

```
public static function getAll(): array {
    $lesObjets = array();
    try {
        $requete = "SELECT * FROM resto";
```

Récupère l'ensemble des enregistrements de la table resto. Chaque ligne est convertie en objet Resto et ajoutée à une liste.

```php
public static function getTop4(): array {
    $lesObjets = array();
    try {
        $requete = "SELECT SUM(note) AS NotesCumulees, r.idR, nomR, numAdrR, voieAdrR, cpR, villeR, latitudeDegR, longitudeDegR, descR, horairesR
                    FROM resto r
                    INNER JOIN critiquer c ON r.idR = c.idR
                    GROUP BY r.idR, nomR, numAdrR, voieAdrR, cpR, villeR, latitudeDegR, longitudeDegR, descR, horairesR
                    ORDER BY NotesCumulees DESC
                    LIMIT 4;
                ";
```

Calcule la somme des notes des critiques associées à chaque restaurant et retourne les 4 restaurants les mieux classés. Les résultats sont instanciés en objets Resto.

```php
public static function getAllByNomR(string $extraitNomR): array {
    $lesObjets = array();
    try {
        $requete = "SELECT * FROM resto WHERE nomR LIKE :nomR";
```

Recherche dans la table resto tous les restaurants dont le nom contient une sous-chaîne donnée (via LIKE). Renvoie une liste d'objets Resto correspondants.

```php
public static function getAllByAdresse(string $voieAdrR, string $cpR, string $villeR): array {
    $lesObjets = array();
    try {
        $requete = "SELECT * FROM resto WHERE voieAdrR LIKE :voieAdrR AND cpR LIKE :cpR AND villeR LIKE :villeR";
        $stmt = Bdd::getConnexion()->prepare($requete);
```

Filtre les restaurants sur la base de leur rue, code postal et ville (tous avec des correspondances partielles grâce à LIKE). Retourne les résultats sous forme d'objets Resto.

```php
public static function getAllMultiCriteres(string $extraitNomR, string $voieAdrR, string $cpR, string $villeR, array $tabIdTC): array {
    $lesObjets = array();
    try {
        if (count($tabIdTC) > 0) {
            $filtre = "idTC = $tabIdTC[0] ";
            for ($i = 1; $i < count($tabIdTC); $i++) {
                $filtre .= " OR  idTC = $tabIdTC[$i] ";
            }
            $requete = "SELECT DISTINCT r.* "
                    . " FROM resto r "
                    . " INNER JOIN proposer p ON r.idR = p.idR "
                    . " WHERE (" . $filtre . ") "
                    . " OR nomR LIKE :nomR"
                    . " OR  voieAdrR LIKE :voieAdrR AND cpR LIKE :cpR AND villeR LIKE :villeR"
                    . " ORDER BY nomR";
```

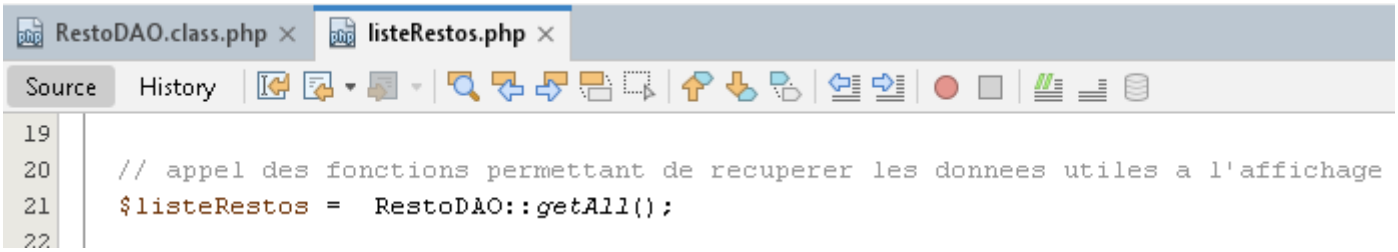
Recherche des restaurants en combinant plusieurs critères (nom, adresse) avec en plus la possibilité de filtrer par types de cuisine. Un restaurant est sélectionné s'il respecte tous les critères texte et au moins un des types de cuisine.

```php
public static function getAimesByIdU(int $idU): array {
    $lesObjets = array();
    try {
        $requete = "SELECT resto.* FROM resto "
                . " INNER JOIN aimer ON resto.idR = aimer.idR"
                . " WHERE idU = :idU";
```

Sélectionne tous les restaurants liés à un utilisateur donné via la table aimer. Retourne ces restaurants sous forme d'objets Resto sans charger leurs critiques ou photos.

2. Quelle méthode de la classe RestoDAO est utilisée dans listeRestos.php ?

C'est la méhode « getAll » de la classe RestoDAO qui est utilisée ici ayant pour but de récupérer tout les restaurants dans la base de données pour les afficher sur le site dans la liste des restos.



III.3 Les couches « vue et « modèle »

L'application web fournie en ressources respecte le « design pattern » MVC ou Modèle Vue Contrôleur. Dans ce patron de conception, les différentes "couches" permettant de construire un site sont gérées de façon indépendante. Découvrons à quoi servent les couches "Vue" et "Modèle".

Répondre aux questions suivantes après avoir observé le contenu des scripts dans les dossiers « controleur » et « vue » du projet.

Travail à faire :

1. Trouve-t-on des éléments de CSS ou de HTML dans les fichiers des dossiers controleur et modele ?

Non, on ne retrouve aucun élément de CSS ou de HTML dans les fichiers des dossiers controleur et modele. Ces fichiers contiennent uniquement du php.

2. Dans quel dossier sont contenus les fichiers produisant du code HTML et CSS ?

Les fichiers produisant du code HTML et CSS sont regroupés dans le dossier « vue » du projet.

3. Le code PHP contenu dans vueListeRestos.php est-il toujours visible après réception par le navigateur ? Par quoi est-il remplacé ?

Non, le code PHP n'est jamais visible par le navigateur une fois la page reçue.
Lors de l'exécution côté serveur, le moteur PHP interprète les instructions (boucles, objets, variables…) et les remplace par le résultat attendu, c'est-à-dire du code HTML (et éventuellement du CSS/JS).

4. Rappeler le point commun entre toutes les méthodes de classe RestoDAO. du dossier modele/dao.

Toutes les méthodes utilisent des requêtes SQL qui interrogent la table **resto** pour extraire des informations sur les restaurants. Elles diffèrent seulement par le type de filtrage ou d'agrégation appliqué (sélection brute, par identifiant, par nom, ou avec calculs statistiques).

5. Trouve-t-on dans d'autres fichiers que ceux du dossier modele/dao des références à la base de données ? Synthèse

Aucun autre fichier ne comprend des références à la base de données en dehors du dossier modele/dao. On en arrive vite à la conclusion que ce dossier est « réservé » au fichiers comprenant ces références pour une meilleure organisation du projet.

6. Expliquer globalement le rôle des scripts contenus dans les dossiers suivants :

◦ modele/dao : Ici, les scripts contiennent principalement tout ce qui est en lien avec la BDD dont les requêtes SQL liées aux différentes méthodes comme les « getAll » ou « getOneById ».

◦ modele/metier : La partie metier en revanche correspond plutôt aux instanciations des classes métier justement pour être ensuite utilisées ailleurs.

◦ vue : Le dossier vue comprend comme son nom l'indique les différentes vues soit les pages html du site qui seront visibles. Ces codes sont l'apparence du site en lui-même.

III.4 La couche contrôleur

III.4.1 - Le contrôleur listeRestos.php

Travail à faire :

1. Quelle méthode DAO est utilisée dans ce contrôleur ? Que permet-elle de récupérer dans la base de données ?

Deux méthode dao sont utiliser, une pour se connecter à la base de donnée : Bdd.class.php
Et une autre qui permet de récupérer les données utiles à l'affichage de la liste des restos : RestoDAO

2. Les données récupérées sont-elles affichées à l'écran ? L'affichage est-il fait dans le contrôleur ?

Oui, elles seront affichées à l'écran, mais pas directement par le contrôleur car Le contrôleur récupère la liste des restaurants dans :
$listeRestos = RestoDAO::getAll();
Ensuite, il charge une vue :
require_once "$racine/vue/vueListeRestos.php";
C'est dans cette vue que la variable $listeRestos sera parcourue et affichée à l'utilisateur
Et non l'affichage n'est pas fait dans le contrôleur il se contente d'appeler la vue

3. Quels scripts sont inclus dans les dernières lignes du contrôleur ? Quels sont leurs rôles ?

Les scripts inclus et leurs rôles :

1. entete.html.php
   - Affiche l'en-tête de la page : ouverture des balises HTML, titre ($titre), menu de navigation, logo…
2. vueListeRestos.php
   - C'est la vue principale appelée par ce contrôleur.
   - Elle affiche concrètement les données récupérées par le DAO ($listeRestos), donc la liste des restaurants.
3. pied.html.php
   - Affiche le bas de page (footer) et ferme correctement les balises HTML (</body>, </html>).

III.4.2 - Le contrôleur detailResto.php
Travail à faire :

1. Quelle donnée est transmise au contrôleur en méthode GET ?

La donnée transmise au contrôleur en méthode GET est idR, c'est-à-dire l'identifiant du restaurant à afficher.

2. Rappeler le rôle de la méthode getOneById. Pourquoi cette méthode a-t-elle besoin d'un paramètre ? La variable $unResto reçoit le résultat de l'appel à getOneById.

C'est une méthode de la classe DAO (RestoDAO).
Elle permet de récupérer un seul restaurant depuis la base de données.
Elle retourne généralement un objet Restaurant si l'ID existe

3. Explorer les fichiers "vue" inclus à la fin du fichier contrôleur et repérer les lignes où cette variable est utilisée.

Dans entete.html.php : la variable unresto n'est pas utiliser
Dans vueDetailResto.php : Partout où on l'on affiche les informations du restaurant
Dans pied.html.php : pas utiliser

4. Le contrôleur gère-t-il directement l'accès aux données ?

Dans ce contrôleur, l'accès aux données n'est pas géré directement, il fait appel à différentes méthodes mises en place dans les DAO pour le faire.

III.4.3 - Synthèse
1. Où sont affichées les données créées ou récupérées dans le contrôleur ? Les données utilisées par le contrôleur peuvent provenir de 2 sources : l'utilisateur ou la base de données.

Dans detailResto.php, la donnée principale est $unResto, obtenue par la méthode getOneById().
→ Ces données **ne sont pas affichées dans le contrôleur** mais transmises à des **vues** (vueDetailResto.php, etc.), qui les affichent au navigateur.

2. Comment ces données sont transmises au contrôleur :

◦ de l'utilisateur vers le contrôleur ?

- L'**identifiant du restaurant (idR)** est envoyé via l'URL en **GET** (index.php?action=detail&idR=...). Le contrôleur le récupère avec $_GET['idR'].

◦ de la base de données vers le contrôleur ?

- Le contrôleur appelle la méthode du modèle $daoResto->getOneById($idR) qui exécute une requête SQL. Le résultat est stocké dans l'objet $unResto.

3. Résumer le rôle de la couche « contrôleur »

Le contrôleur :
- récupère les **données de l'utilisateur** (paramètres GET/POST),

- appelle le **modèle (DAO)** pour interroger la base,
- stocke les données reçues (ex. : $unResto),
- puis les transmet aux **vues** pour l'affichage.
  → Il agit comme **intermédiaire** entre l'utilisateur, la base de données et l'affichage.

IV - Synthèse globale

MVC est l'acronyme de Modèle Vue Contrôleur. Dans le domaine du développement d'applications, MVC est un patron de conception. C'est une bonne pratique de développement d'une application.

Son application apporte plusieurs avantages :
• faciliter le travail en équipe : les composants peuvent être écrits par différentes personnes ;
• faciliter le test unitaire : chacun des composants peut être testé séparément ;
• maintenance et évolutivité : il est possible de revoir le code, ou de changer de technologie sur un des composants (la vue par exemple) sans devoir modifier le reste du code.

Chaque fonctionnalité, par exemple l'affichage de la liste des restaurants, fait ainsi appel aux composants :
• Modèle : fonction d'accès à la base de données ;
• Vue : affichage des données à l'utilisateur ;
• Contrôleur : composant chargé de la logique applicative : récupération les données saisies par l'utilisateur, appel des fonctions du modèle, traitement des données, puis appel des vues pour l'affichage.

Le patron de conception MVC contraint et guide le programmeur dans sa manière de coder une application. Il lui impose des règles, mais en retour il permet de faciliter la maintenance, et le travail en équipe.
Principes de base du MVC :
• le contrôleur et le modèle ne procèdent à aucun affichage : c'est le rôle de la vue ;
• les requêtes SQL sont exclusivement situées dans les méthodes de la couche DAO ;
• les vues ne doivent contenir que du code d'affichage de données provenant du contrôleur, ou éventuellement du modèle (des classes métier).